

Taming Rollout Risks in Distributed Web Apps

A Location-Aware Gradual Deployment Approach

Presented by: Angela Victorio
KubeCon 2025 North America

A decorative light blue triangle is located in the bottom right corner of the slide, pointing towards the top right.

Why Gradual Rollout?

- Kubernetes minimizes downtime, but production can reveal bugs
- Gradual rollout limits exposure and reduces blast radius
- Goal: Make releases boring and predictable

Gradual Rollout Strategies

- **Feature Flags:** App-level control, requires feature management
- **Cluster-by-Cluster:** Ideal for multi-cluster environments
- **DNS-based Traffic Splitting:** Coarse control via DNS records
- **Service Mesh (Istio):** Flexible, powerful, demo focus

Demo Setup

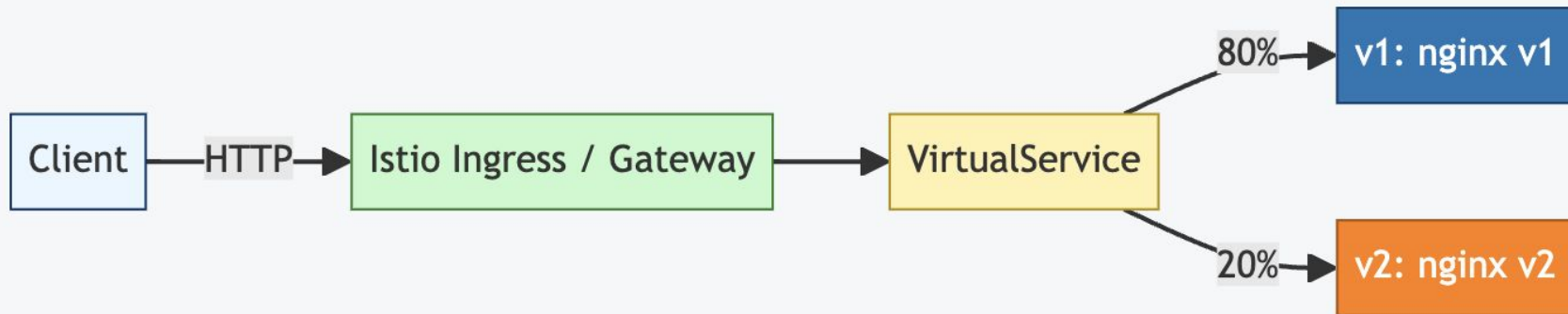
- Local Kind cluster with Istio installed
- Monitoring: Kiali or Kubernetes dashboard
- Demo repos on GitHub:
 - Demo 1: github.com/bretonne/demo-gradual-rollout
 - Demo 2: github.com/bretonne/location-rollout-demo
 -
- Follow README to run scripts and explore code
- Note: AI-generated scripts for demo purposes only

Demo 1 — Percentage-based Rollout

- Split traffic between v1 and v2 using Istio VirtualService weights

Ideal for random sampling across global traffic

Example: Shift weights (90/10 → 50/50 → 0/100)

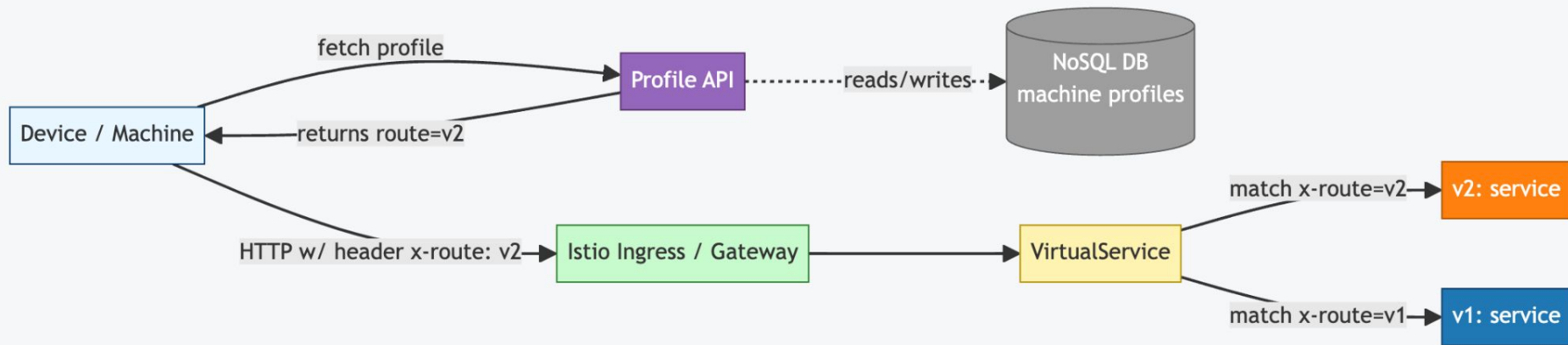


Demo 2 – Location-based Rollout

- Uses Istio routes with request headers set by client-side logic

Problem: Small sites may be over/under-represented in random samples

Solution: Group devices by location, apply percentage per location



Monitoring & Rollback

- Instrument both v1 and v2 with metrics and logs
- Monitor error rate, latency, and SLOs per location
- Rollback: Revert 100% to previous version if degradation detected

“The foundation of a robust rollback strategy starts well before the release is ever deployed.”

- Manifestly

Practical Tips

- **Timing:** Rollouts may be slower, plan accordingly
- **Decouple changes:** DB schema, service, and frontend (ensure backward compatibility)
- **Backward Compatibilities:** Read more: [Strategies for Backward Compatibility](#)

Closing

- Gradual rollouts reduce risk and ensure stable release
- Istio enables flexible traffic management (percentage, location-based)
- Monitor closely and prioritize backward compatibility
- Try the demos: [Demo 1](#), [Demo 2](#)

Questions?



References

<https://kubernetes.io/>

<https://istio.io/>

<https://www.manifest.ly/use-cases/software-development/rollback-plan-checklist>